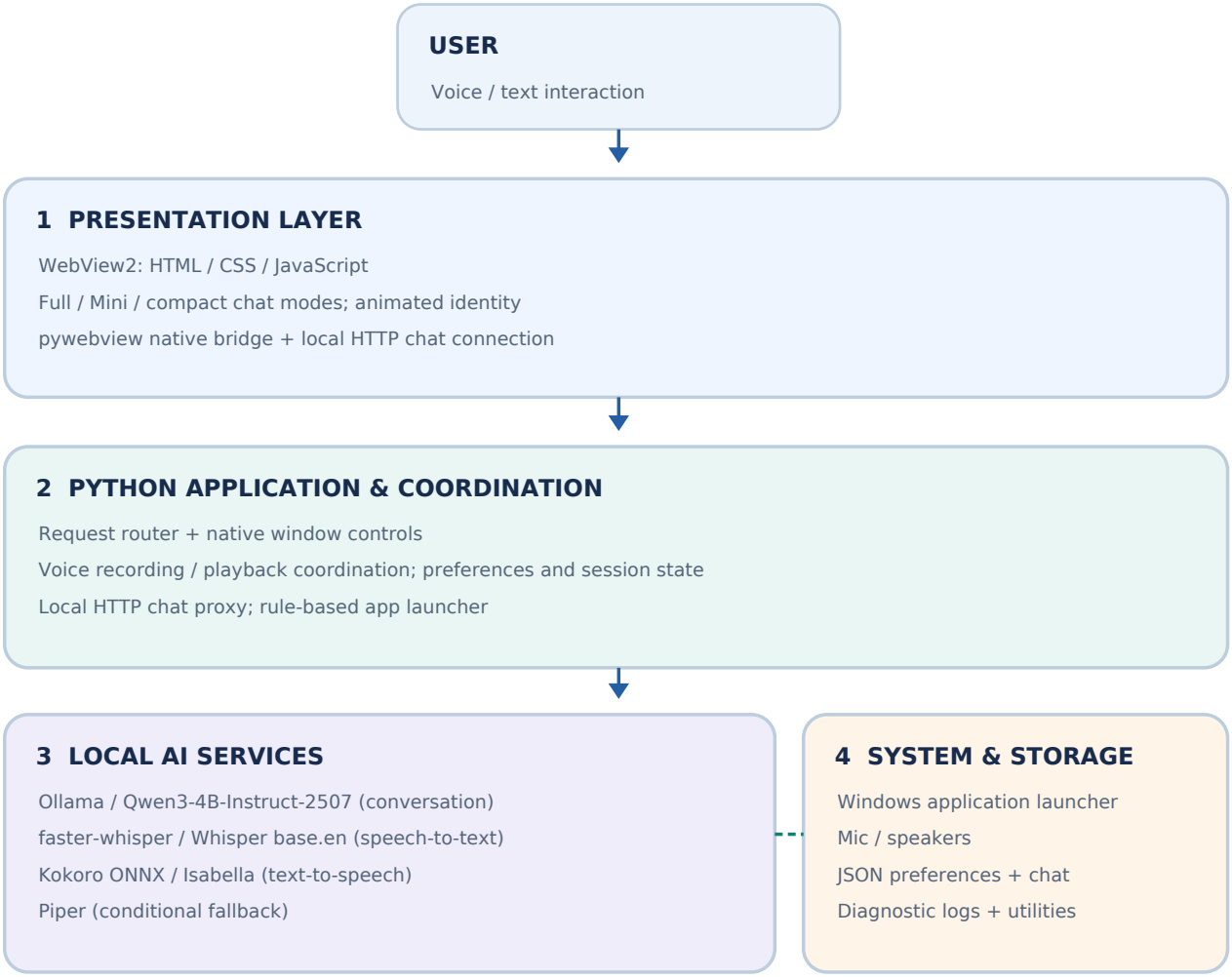
ARTEMIS | System Architecture

Professor presentation brief • Offline-first Windows desktop AI assistant

ARTEMIS combines local language generation, speech recognition, speech synthesis and controlled Windows application actions in a modular desktop interface. Online access is required only for supporting external-information utilities.

High-level layered architecture



Conceptual component grouping based on the supplied ARTEMIS architecture document.

Key design principle. Presentation, coordination, local AI inference and operating-system integration are separated so components can be maintained or replaced independently. The language model is not permitted to generate and directly execute arbitrary shell commands.

Request Processing & AI Stack

End-to-end processing

1. User types text or begins a voice turn after explicit activation.

2. Voice input is transcribed by faster-whisper (Whisper).

3. Python routing checks whether an explicit tool or application command is supported.

4. Supported commands execute through deterministic services; other requests use localhost server → Ollama → Qwen3.

5. Results appear in the interface and conversation history is saved locally.

6. When enabled, Kokoro / Isabella speaks the response; Piper may provide fallback.

7. Hands-free mode starts another turn after spoken playback has finished.

Core AI models and runtimes

Component	Model / runtime	Purpose
Local LLM	Qwen3-4B-Instruct-2507 via Ollama (`artemis`)	Conversational responses
Speech-to-text	Whisper base.en via faster-whisper	Transcribe English speech
Primary text-to-speech	Kokoro v1.0 / bf_isabella via kokoro-onnx	Spoken responses
Fallback text-to-speech	Piper (local voice if available)	Resilience if Kokoro unavailable

Model configuration. Qwen model source is configured as kamekichi128/qwen3-4b-instruct-2507; project ModelFile specifies temperature 0.7 and context 4096. Whisper defaults to CPU / int8, with supported optional CUDA / float16 settings. The active Ollama manifest has not been verified by this document.

Important distinction. ARTEMIS integrates pretrained AI models; it is not a newly trained foundation model. The pause-based end-of-turn detector uses audio-energy/timing rules, not a wake-word model.

Functions, Boundaries & Modules

Main capabilities

Conversation and context. Local Ollama chat server, bounded recent-message context and locally saved chat history; not RAG or trained long-term memory.

Voice and hands-free. Microphone via sounddevice; faster-whisper ASR; Kokoro speech. Hands-free requires explicit activation; no always-on wake word is required.

Controlled Windows actions. Rule-based application discovery, name matching and launching; ambiguous app matches require confirmation.

Desktop interface. Borderless WebView2 UI, full/Mini/compact modes, native resizing and pinning, animated activity states.

Privacy and reliability. Local models and JSON persistence, asynchronous audio/network workers, atomic JSON replacement and diagnostic logging. Some utilities use internet services.

Representative implementation modules

desktop/main.py	Native host, bridge and coordination
desktop/server.py	Local chat server / Ollama proxy
desktop/web/	Desktop UI and interaction modes
desktop/whisper_input.py	Audio input / transcription
desktop/speech_endpoint.py	Pause-based end-of-turn
desktop/speech_output.py + artemis/voice.py	Speech playback / Kokoro / Piper
desktop/app_launcher.py	Rule-based Windows app commands
desktop/desktop_state.py	Preferences / chat persistence

Academic discussion notes

Engineering contribution: Coordinated local inference, speech processing, deterministic desktop controls and responsive UI. Limitations: external information tools need network access; local conversation files are not encrypted; model runtime defaults are not benchmark results. Suggested live demo: typed prompt → local response; explicit “open Notepad” → validated app launch; voice turn → transcription → spoken response.